

Test Summary Report

****Course:**** Software Testing and Validation

****Group members:**** Eyob Mulugeta (ATE/4778/14); Afomia Birhanu (ATE/2368/14)

1. Summary of Testing Activities

A complete testing effort was executed on the Vehicle Rental System. Testing utilized the Test Pyramid approach:

- * ****Unit Tests**:** Focused heavily on the core ``RentalCalculator`` (Age, Duration rules, Discount logic via Decision Tables) and ``BookingManager`` (State Transitions).
- * ****Integration Tests**:** Validated the interaction between the Django ORM, the SQLite database, and the business logic classes.
- * ****System and acceptance tests**:** Automated UI tests using Playwright and the Page Object Model (POM) pattern to verify successful and rejected booking journeys.

2. Results Against Exit Criteria

- * ****Test Execution**:** The local baseline executed ****27 tests: 27 passed****. The same command is configured for GitHub Actions and Jenkins; hosted build links/screenshots must be attached after execution.
- * ****Code Coverage**:** The measured baseline achieved ****94% total coverage**** and ****100% branch coverage for ``rental/business_logic.py``****. (Pass)
- * ****Defects**:** 4 defects were identified and resolved. 0 high-severity defects remain open. (Pass)

3. Outstanding Defects & Residual Risk

There are currently no known outstanding defects. The residual risk is considered ****Low****. Given the high code coverage and successful execution of end-to-end paths, the primary remaining risks involve production environment configuration (e.g., PostgreSQL instead of SQLite) which was outside the scope of this project.

4. Release Recommendation

****Approved for Release.**** The application meets all defined quality thresholds and functional requirements.

5. Demonstrated Regression (Part E)

During development, a regression was deliberately introduced to verify the continuous integration pipeline's ability to catch it.

- * ****The Change**:** In ``rental/business_logic.py``, the ``YOUNG_DRIVER_SURCHARGE`` was incorrectly altered from ``20.0`` to ``15.0``.
- * ****The Failing Build**:** The GitHub Actions or Jenkins run must be linked or shown in an attached screenshot after the intentional change. The expected failure is ``test_age_at_minimum_with_surcharge``, where \$350.0 becomes \$325.0.
- * ****The Fix**:** Revert the value to ``20.0`` and rerun the pipeline. Attach the passing build as evidence. This workspace is not currently a Git checkout, so commit and hosted-build evidence must be created in the group's remote repository.

Part I: Foundations Reflection

****Defect Analyzed**:** DEF-001 (30-day duration throws error)

During development, ``RentalCalculator.calculate_price`` threw a ``DurationRestrictionError`` when a user attempted to book a vehicle for exactly 30 days.

- * ****The Error (Human Action)**:** The developer misunderstood the requirements for the maximum rental period boundary. They typed ``>= 30`` instead of ``> 30`` in the conditional logic.
- * ****The Fault (The Code Defect)**:** The code contained the statement ``if duration_on_days >= 30: raise DurationRestrictionError``.

* ****The Failure (The Observable Result)**:** When running the unit tests (specifically the Boundary Value Analysis test case `TC\_Dur\_03` which passed exactly `30`), the test runner crashed with the unexpected exception.

****Verification vs Validation**:**

This defect was caught by ****Verification****. The automated unit tests verified that the code met the documented specification (which stated 1 to 30 days is valid). It was caught early in the development lifecycle before any user (Validation) ever interacted with the system.